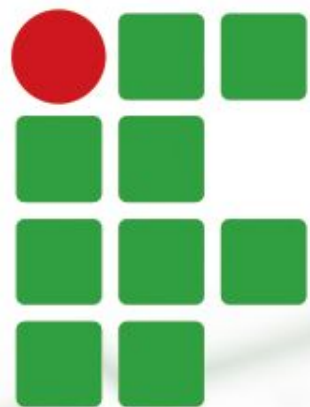




INSTITUTO FEDERAL

Norte de Minas Gerais

Campus Januária

Estruturas de Dados 2

- Function Pointers -



Function Pointers

■ Observe que interessante...

```
#include <stdio.h>

int soma(int a, int b){
    printf("Realizando Soma %d + %d == ",a,b);
    return a+b;
}

int prod(int a, int b){
    printf("Realizando Produto %d x %d == ",a,b);
    return a*b;
}
```

Temos duas funções (**soma** e **prod**) que calculam operações matemáticas, mas sem apresentar o resultado final...



Function Pointers

■ Observe que interessante...

```
#include <stdio.h>
```

```
void calcular(int (*func)(), int a, int b){  
    int res = func(a,b);  
    printf("%d\n",res);  
}
```

A função **calcular** recebe como argumento, um **ponteiro para função** (*Pointer Function*) que será executada internamente, adaptando assim o seu comportamento.

```
int prod(int a, int b){  
    printf("Realizando Produto %d x %d == ",a,b);  
    return a*b;  
}
```



Function Pointers

■ Observe que interessante...

```
#include <stdio.h>

void calcular(int (*func)(), int a, int b){
    int res = func(a,b);
    printf("%d\n",res);
}

int main(int argc, char const *argv[]) {
    calcular(soma,2,5);
    calcular(prod,2,5);
}
```

Assim, quando invocamos a função **calcular**, devemos enviar a função (ponteiro) que desejamos que ela execute internamente.



Function Pointers

- Observe que interessante...

```
#include <stdio.h>

int main() {
    printf("Realizando Soma 2 + 5 == 7\n");
    printf("Realizando Produto 2 x 5 == 10\n");
    return 0;
}
```

Terminal

Arquivo Editar Ver Pesquisar Terminal Ajuda

Realizando Soma 2 + 5 == 7
Realizando Produto 2 x 5 == 10

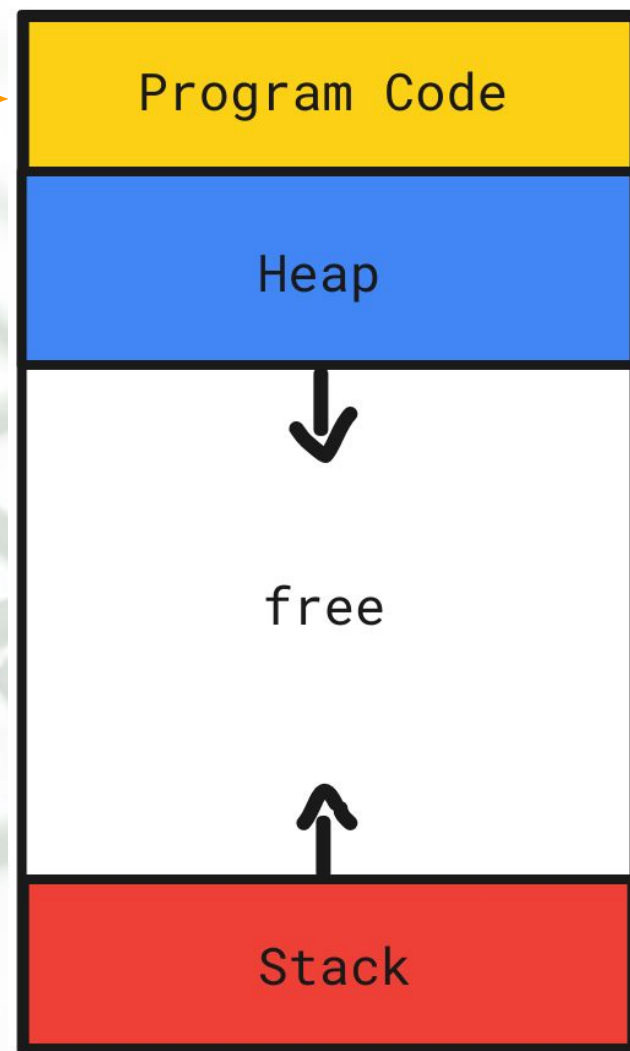
(program exited with code: 0)
Press return to continue



Function Pointers

Function Pointer

(Ponteiro para Função) é um ponteiro (endereço de memória) que **faz referência a uma instrução executável**, em vez de dados, como nos ponteiros de variáveis já conhecidos.





Function Pointers

- É um recurso muito útil para definir, **em tempo de execução, qual rotina que deverá executada**, sob supervisão de quem a invoca.
P.Ex.: Callbacks, Plug-ins, OOP, etc...
- **Função de Ordem Superior** é o nome dado às funções que recebem *function pointers* como argumento.
- Vamos ver mais um exemplo prático...



Function Pointers

■ Exemplo de *Callback* com *Function Pointer*

```
void tarefa_demorada(void (*callback)()){  
    //...  
    //rotina demorada  
    //...  
    callback();  
}  
  
int main(){  
    tarefa_demorada(send_Email);  
    //tarefa_demorada(send_ZAP);  
    //tarefa_demorada(ring_Alarm);  
    //tarefa_demorada(do_Nothing);  
}
```




Function Pointers

■ Exemplo de *Callback* com *Function Pointer*

```
void tarefa_demorada(void (*callback)()){  
    //...  
    //rotina demorada  
    //...  
    callback();  
}
```

Função de Ordem Superior, ou seja, uma função que recebe como argumento um ponteiro para função.

```
int main(){  
    tarefa_demorada(send_Email);  
    //tarefa_demorada(send_ZAP);  
    //tarefa_demorada(ring_Alarm);  
    //tarefa_demorada(do_Nothing);  
}
```



Function Pointers

■ Exemplo de *Callback* com *Function Pointer*

```
void tarefa_demorada(void (*callback)()){  
    //...  
    //rotina demorada  
    //...  
    callback();  
}  
  
int main(){  
    tarefa_demorada(send_Email);  
    //tarefa_demorada(send_ZAP);  
    //tarefa_demorada(ring_Alarm);  
    //tarefa_demorada(do_Nothing);  
}
```

Function Pointer. Ponteiro para uma função que tem retorno `void`, e que não possui parâmetros. Internamente essa função se chamará `callback()`



Function Pointers

■ Exemplo de *Callback* com *Function Pointer*

```
void tarefa_demorada(void (*callback)()){  
    //...  
    //rotina demorada  
    //...  
    callback();  
}
```

Invocação da Function Pointer recebida
como argumento.

```
int main(){  
    tarefa_demorada(send_Email);  
    //tarefa_demorada(send_ZAP);  
    //tarefa_demorada(ring_Alarm);  
    //tarefa_demorada(do_Nothing);  
}
```



Function Pointers

■ Exemplo de *Callback* com *Function Pointer*

```
void tarefa_demorada(void (*callback)()){  
    //...  
    //rotina demorada  
    //...  
    callback();  
}
```

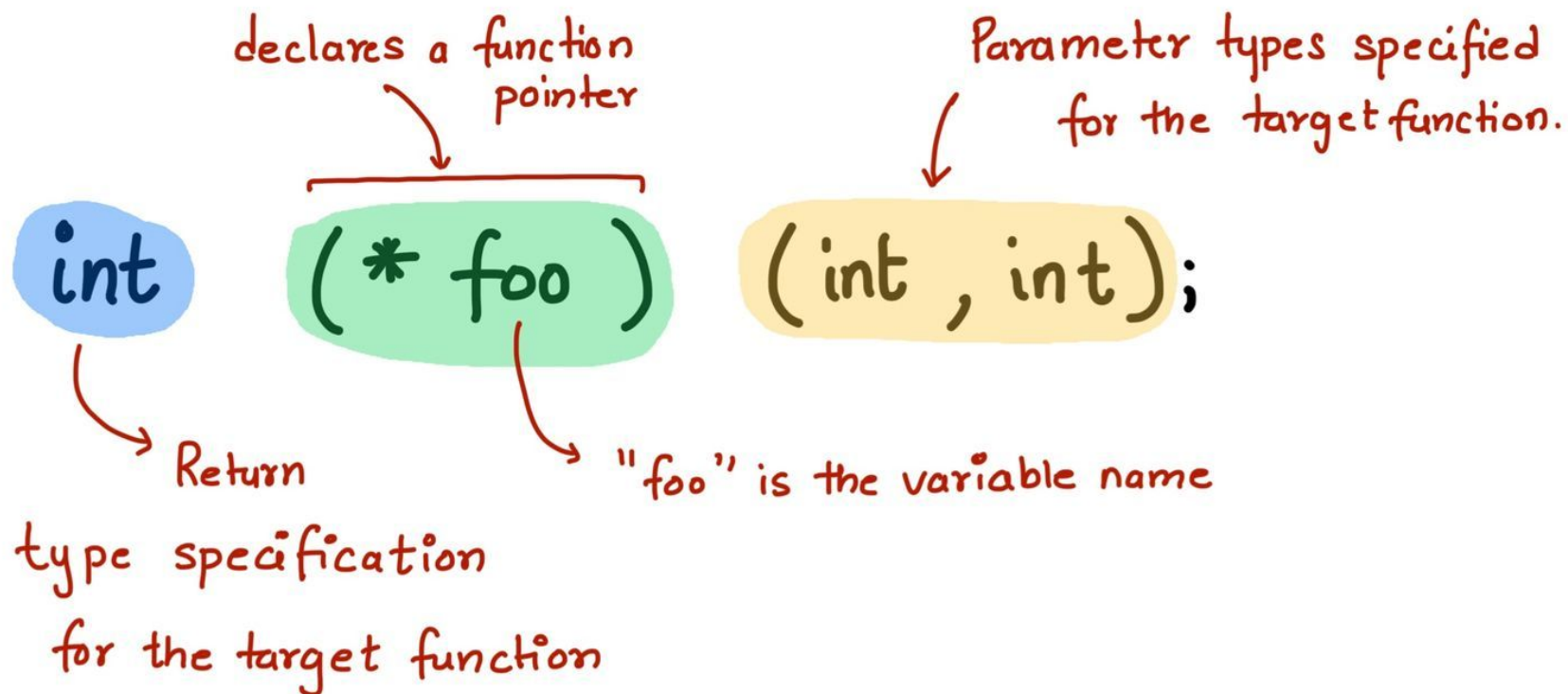
Funções que poderiam ser enviadas como argumento para a Function Pointer. Essas funções devem estar implementadas em algum lugar do código.

```
int main(){  
    tarefa_demorada(send_Email);  
    //tarefa_demorada(send_ZAP);  
    //tarefa_demorada(ring_Alarm);  
    //tarefa_demorada(do_Nothing);  
}
```



Function Pointers

■ Declaração de *Function Pointers*...





Function Pointers

■ Declaração de *Function Pointers*...

```
tipo_retorno (*nome_funcao) ( )
```

- **Tipo** de dado que a função retornará.
- **Nome** que a função assumirá internamente.
- **Tipos dos Parâmetro(s)** necessários para a execução da função.

```
void ordem_superior(int(*func)())
```

```
void ordem_superior(void(*send)(char*), char* dest)
```

```
void ordem_superior((*Pessoa)(*get)(Pessoa[],int), int v)
```




Function Pointers

■ Declaração de *Function Pointers*...

```
tipo_retorno (*nome_funcao) ( )
```

- **Tipo** de dado que a função retornará.
- **Nome** que a função assumirá internamente.
- **Tipos dos Parâmetro(s)** necessários para a execução da função.

```
void ordem_superior(int(*func)())
```

```
void ordem_superior(void(*send)(char*), char* dest)
```

```
void ordem_superior((*Pessoa)(*get)(Pessoa[],int), int v)
```

ATENÇÃO! As funções enviadas como argumento devem respeitar a assinatura requerida pela Function Pointer!



O Poder das Function Pointers

■ Plugins

```
void tarefa_demorada(void (*callback)()){  
    //...  
    //rotina demorada  
    //...  
    callback();  
}
```

Array de Function Pointers para oferecer flexibilidade/customização ao código.

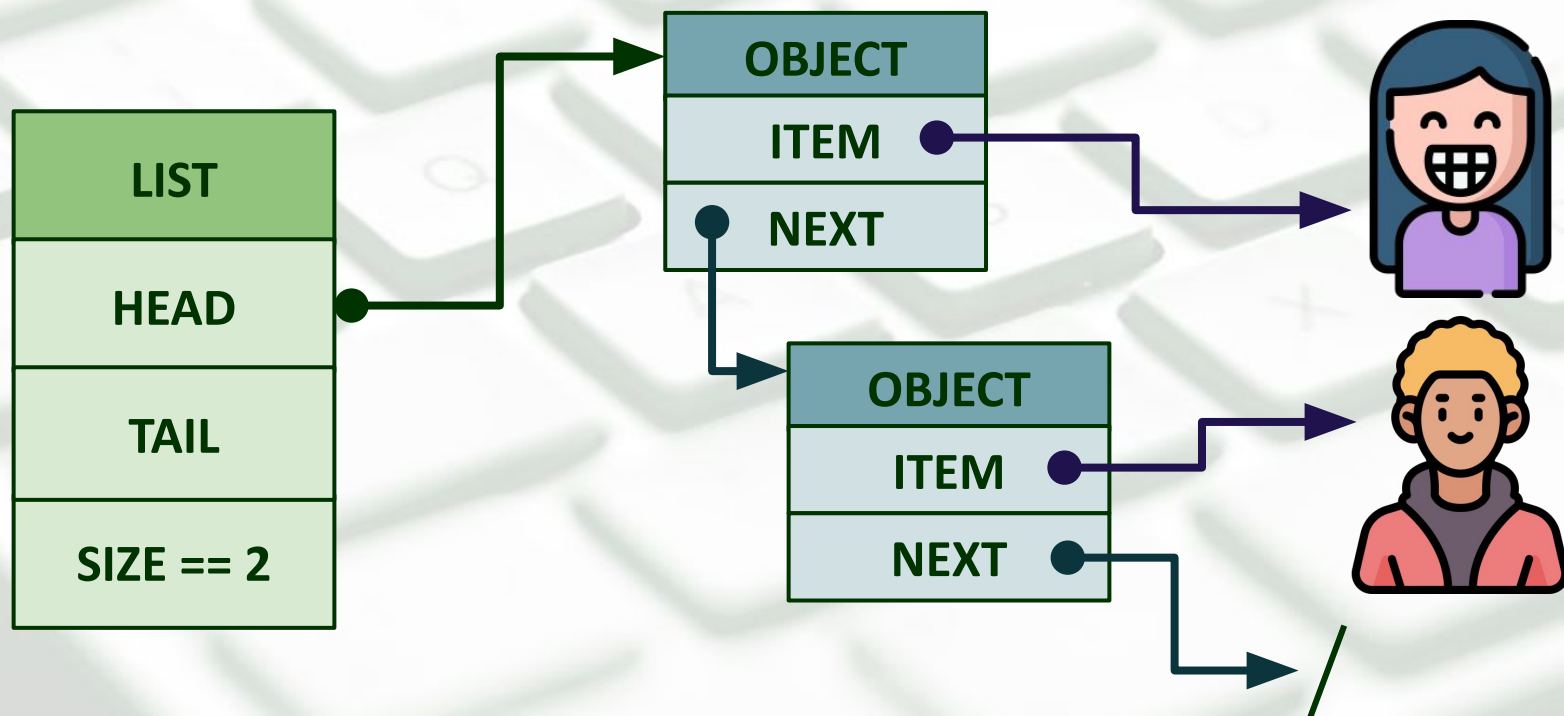
Novos alertas podem ser facilmente integrados ao código (plug-ins)

```
int main(){  
    void (*alerts[3])() = {send_Email,send_ZAP,ring_Alarm};  
    int option = input("Como deseja ser notificado?");  
    tarefa_demorada(alerts[option]);  
}
```



2º Desafio

- VOLTANDO AO NOSSO PROBLEMA ANTERIOR...
Como os itens de uma Lista Genérica são “indefinidos”, como executar funções específicas (p.ex. Impressão dos dados)





Function Pointers

```
int main() {  
    List lst_pessoas = new(List);  
    List lst_produtos = new(List);  
  
    do{  
        switch(interface()){  
            case 3: list_print(lst_pessoas, print_Pessoa);  
                    break;  
            case 4: list_print(lst_produtos, print_Produto);  
                    break;  
            case 0: return 0;  
                    break;  
        }  
    }while(1);  
}
```

*Listas Específicas para
cada tipo de entidade...*



Modelo em Java

■ Exemplo de implementação em Linguagens de alto nível

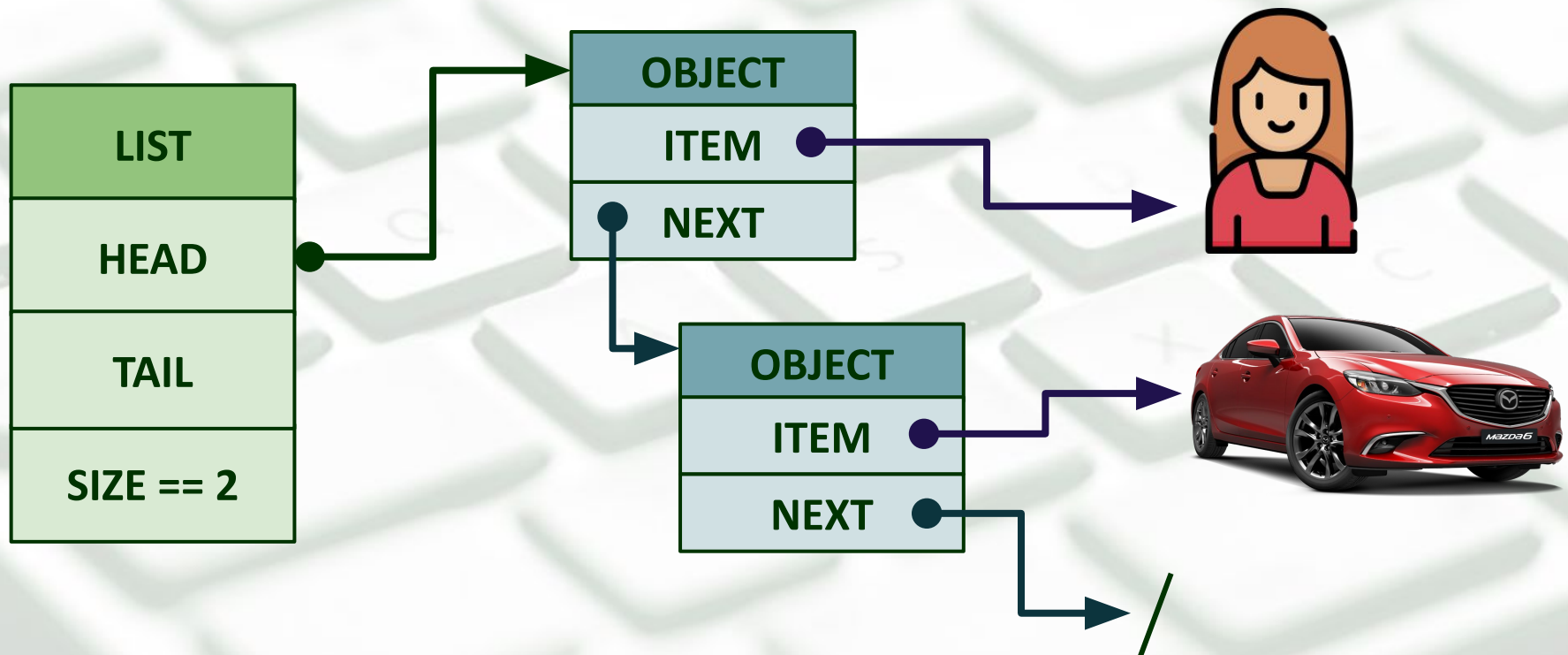
```
import java.util.LinkedList;

public class Main{
    public static void main(String[] args) {
        LinkedList<Carro> cars = new LinkedList<Carro>();
        cars.add(new Carro("Hilux", "Toyota"));
        cars.add(new Carro("Toro", "Fiat"));
        cars.add(new Carro("T-Cross", "VW"));
        cars.add(new Carro("Territory", "Ford"));
        System.out.println(cars);
    }
}
```



Listas Heterogêneas

- Mas e se a Lista for **HETEROGÊNEA**?





Modelo em Python

■ Exemplo de implementação em Linguagens de alto nível

```
def main():  
    lst = []  
    lst.append(Carro("Hilux", "Toyota"))  
    lst.append(Pessoa("João", 30))  
    lst.append(Carro("T-Cross", "VW"))  
    lst.append(Pessoa("Maria", "25"))  
  
    print(list(lst))
```



O Poder das Function Pointers

```
typedef struct{  
    char nome[100];  
    char raça[100];  
    int hp, xp;  
    int (*atacar)(int);  
    void (*andar)(char);  
    int (*morrer)();  
}_Personagem;  
  
typedef _Personagem* Personagem;
```

Function Pointers como
membros de structs



O Poder das Function Pointers

```
typedef struct{
    char nome[100];
    char raça[100];
    int hp, xp;
    int (*atacar)(int);
    void (*andar)(char);
    int (*morrer)();
}_Personagem;

typedef _Personagem* Personagem;
```

Function Pointers como
membros de structs

```
int main(){
    Personagem p = new(Personagem, "Legolas", "Elfo", 1000, 80);
    //...
    p->andar(p, 'd');
    p->atacar(p, 5);
}
```

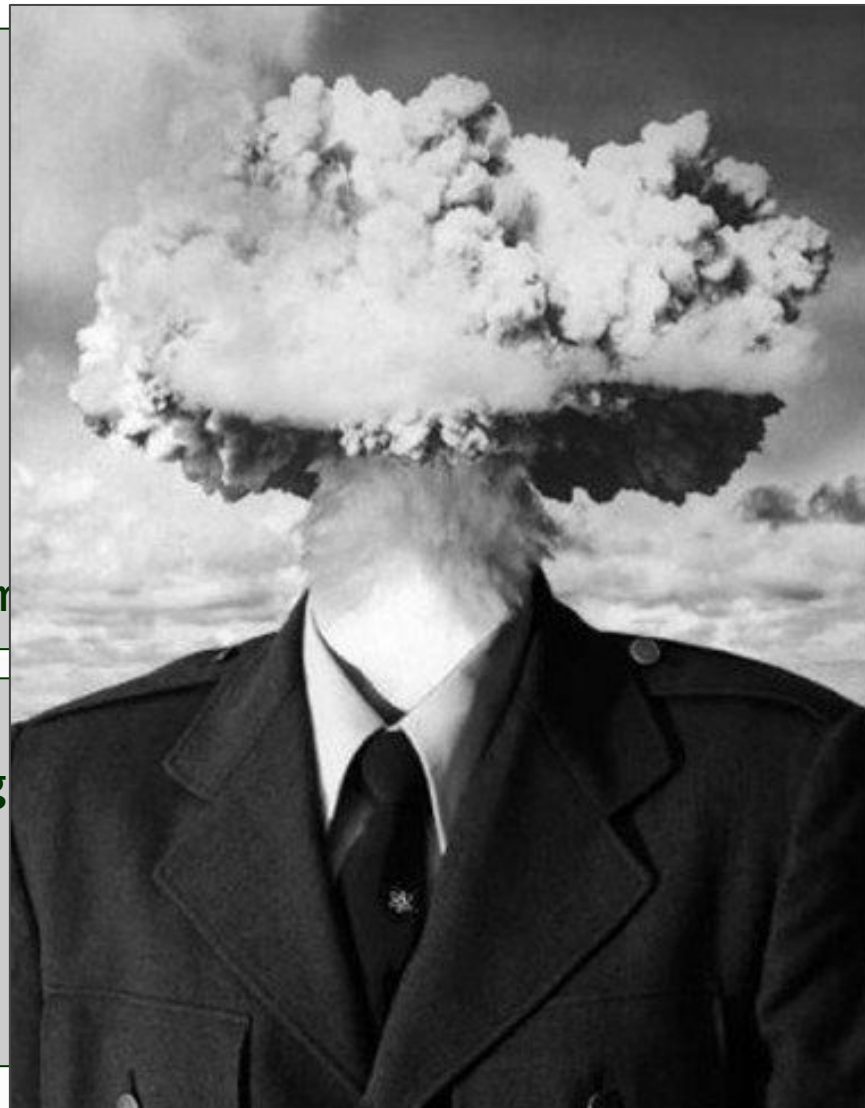
Em que isso se parece???



O Poder das Function Pointers

```
typedef struct{  
    char nome[100];  
    char raça[100];  
    int hp, xp;  
    int (*atacar)(int);  
    void (*andar)(char);  
    int (*morrer)();  
}_Personagem;  
  
typedef _Personagem* Personagem
```

```
int main(){  
    Personagem p = new(Personagem)  
    //...  
    p->andar(p, 'd');  
    p->atacar(p, 5);  
}
```





O Poder das Function Pointers

```
class Personagem():
    def __init__(self, nome, raca, hp, xp):
        self.nome = nome;
        self.raca = raca;
        self.hp = hp;
        self.xp = xp;

    def atacar(self, forca):
        print(f'{self.nome} está atacando com forca {forca}.');

    def andar(self, direcao):
        print(f'{self.nome} está andando para a {direcao}.');

    def morrer(self):
        print(f'{self.nome} morreu.');
```

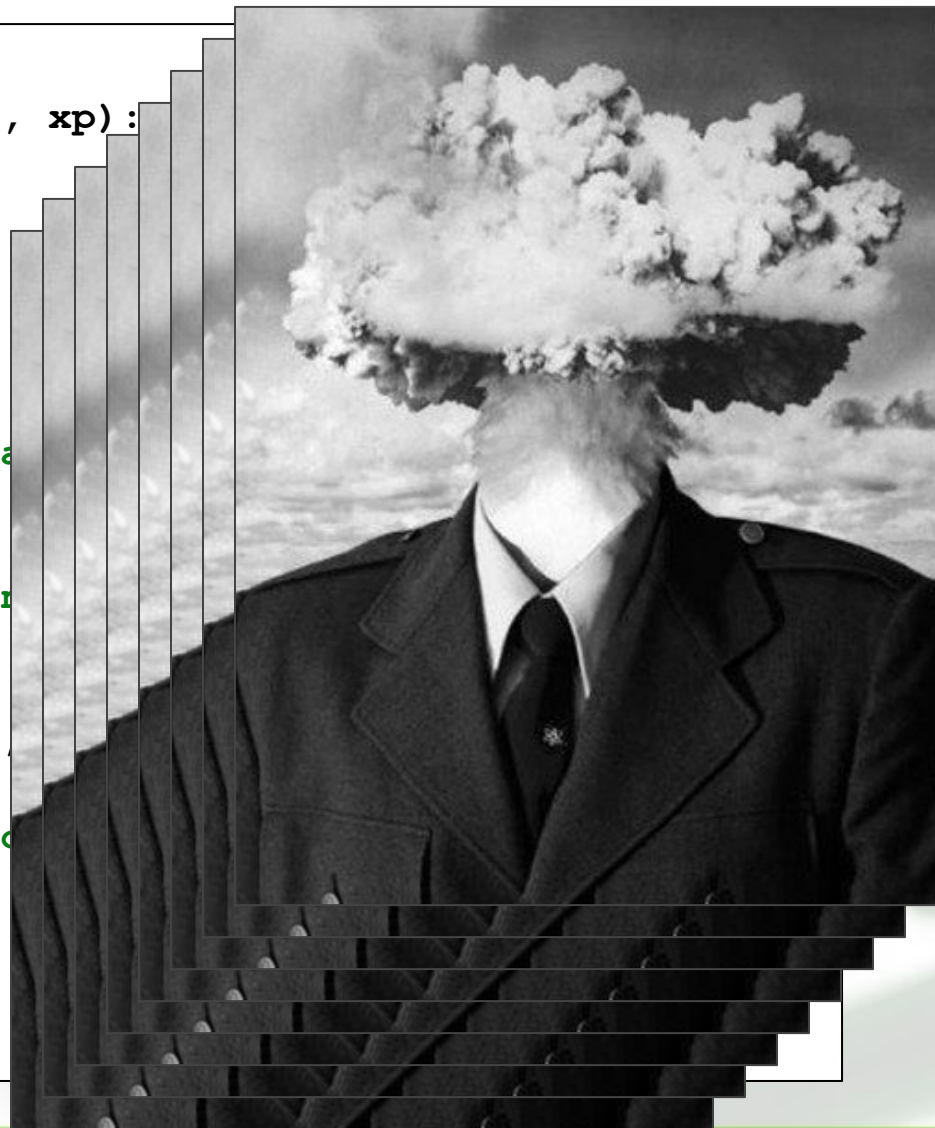


```
legolas = Personagem("Legolas", "Elfo", "100", "20")
legolas.andar('d')
legolas.atacar(5)
legolas.morrer()
print(Personagem.mro())
```




O Poder das Function Pointers

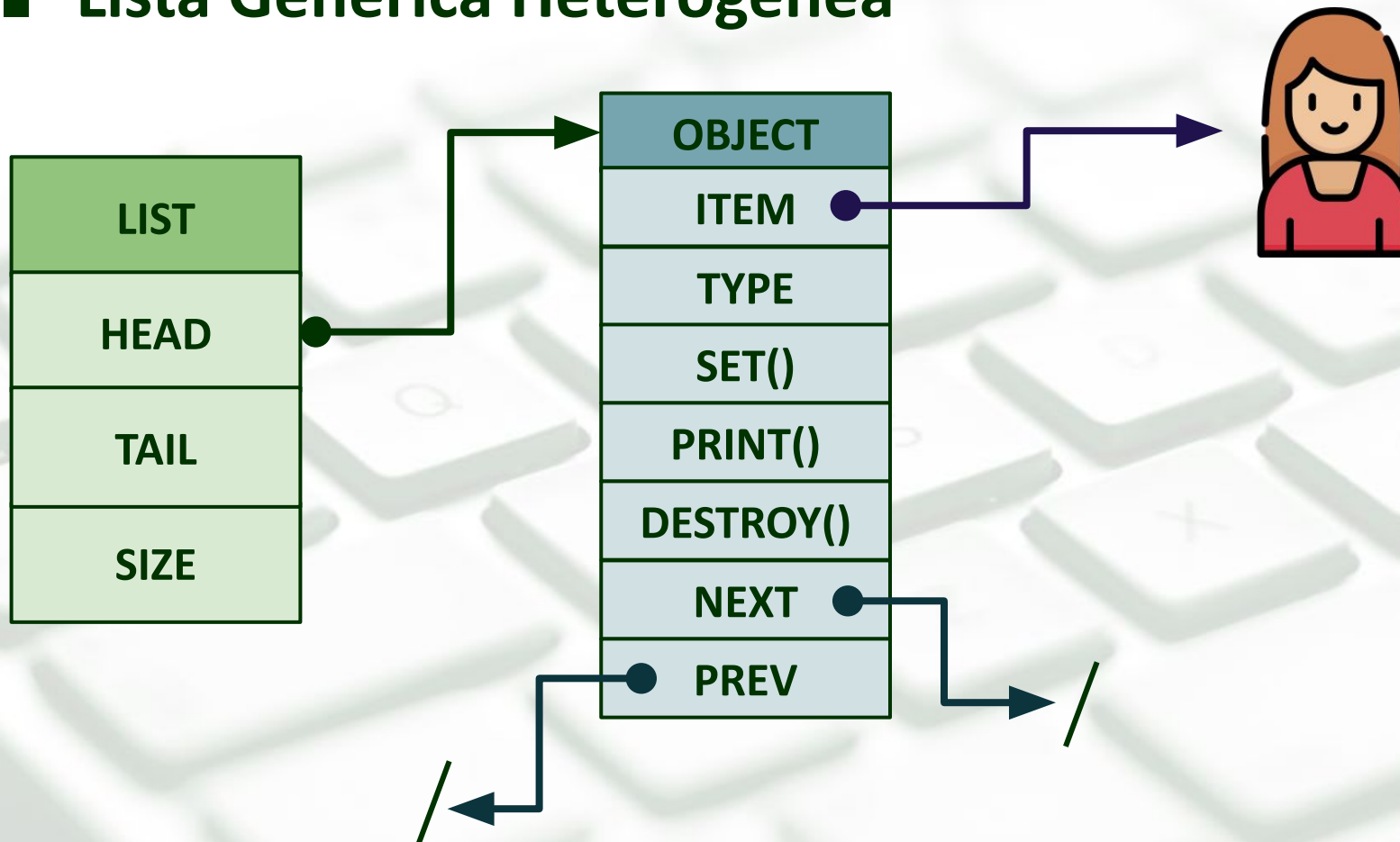
```
class Personagem():  
    def __init__(self, nome, raca, hp, xp):  
        self.nome = nome;  
        self.raca = raca;  
        self.hp = hp;  
        self.xp = xp;  
  
    def atacar(self, forca):  
        print(f'{self.nome} está atacando')  
  
    def andar(self, direcao):  
        print(f'{self.nome} está andando')  
  
    def morrer(self):  
        print(f'{self.nome} morreu.')  
  
legolas = Personagem("Legolas", "Elfo")  
legolas.andar('d')  
legolas.atacar(5)  
legolas.morrer()  
print(Personagem.mro())
```





Projeto de Implementação

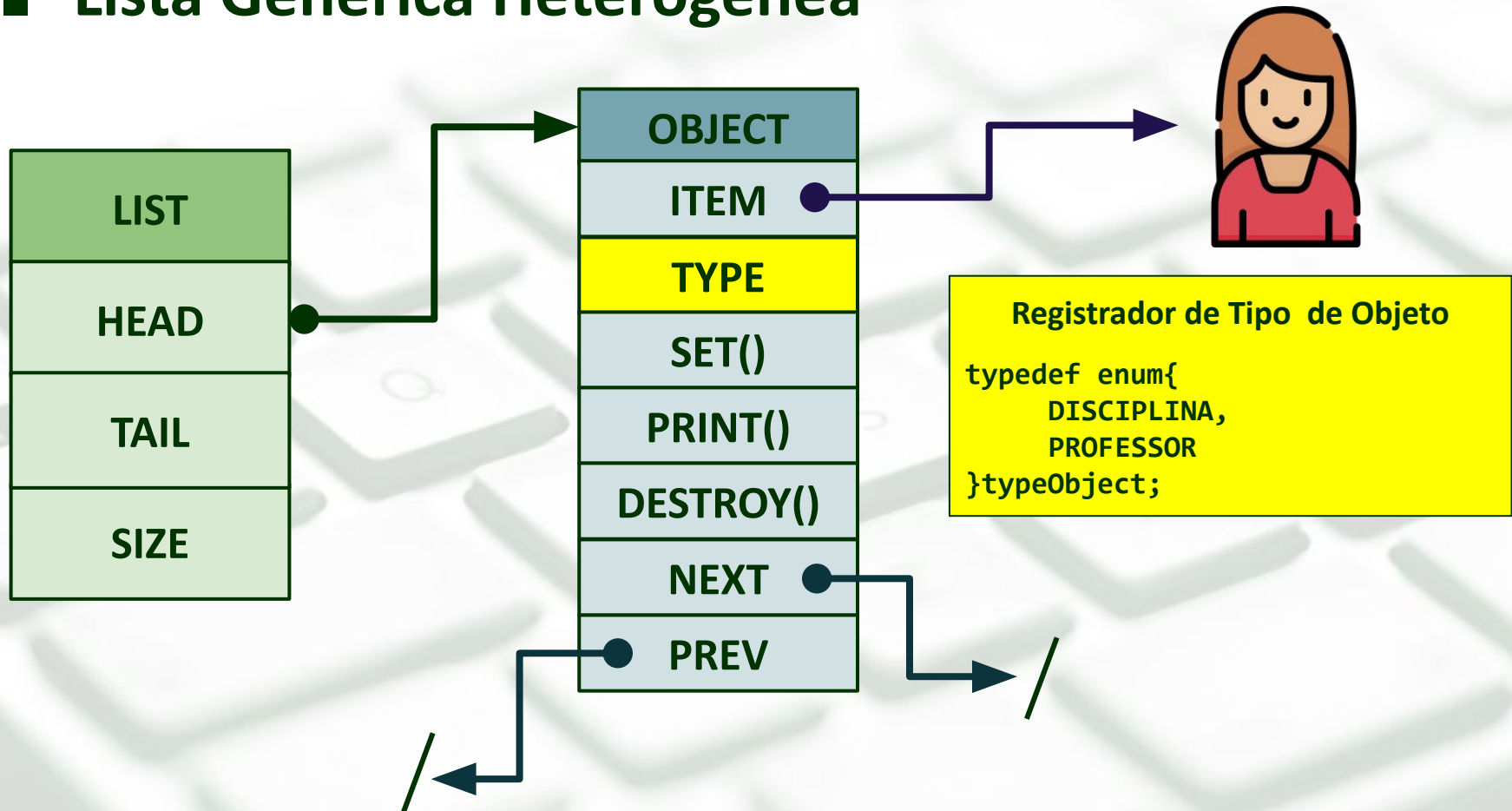
■ Lista Genérica Heterogênea





Projeto de Implementação

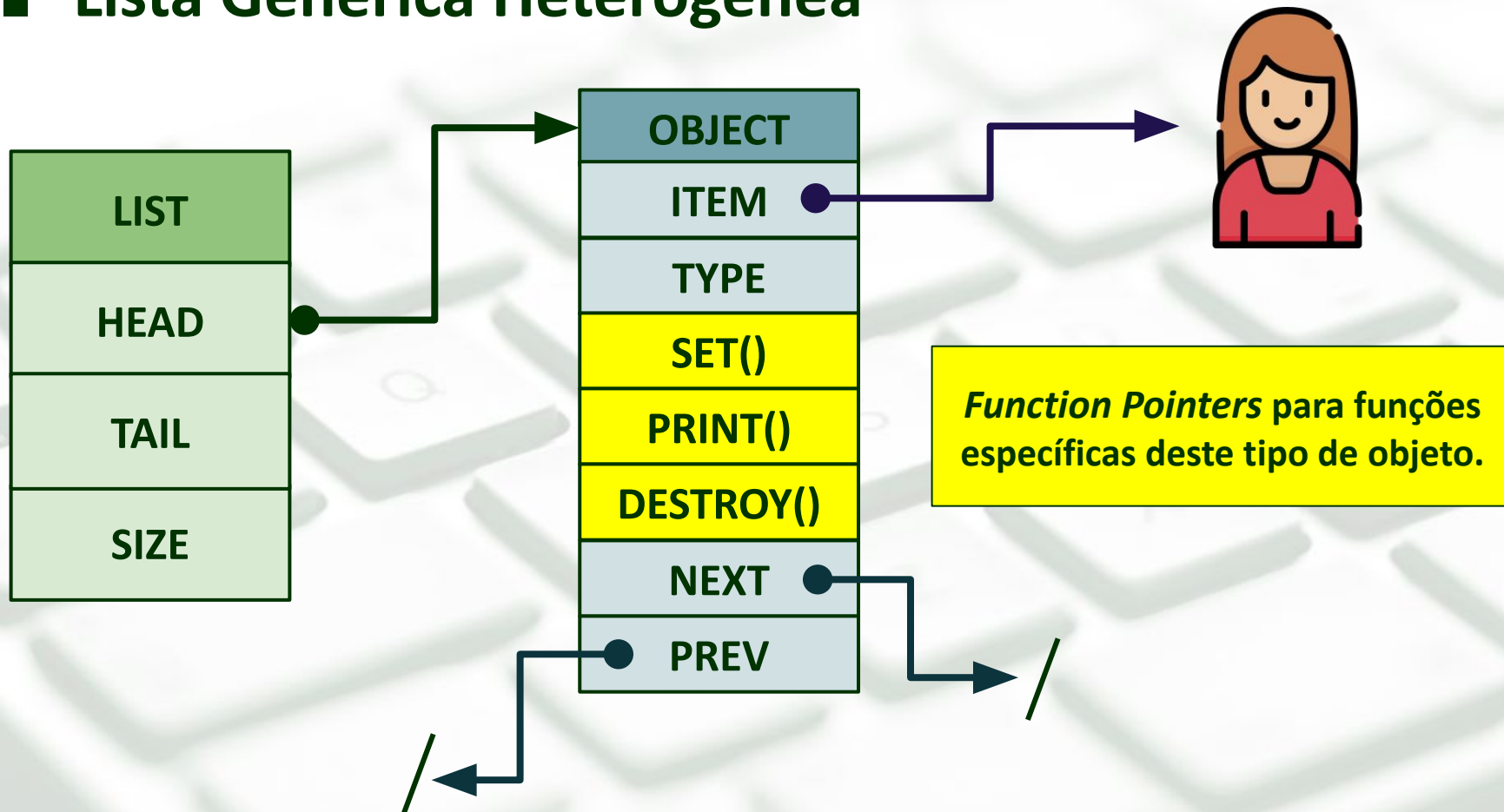
■ Lista Genérica Heterogênea





Projeto de Implementação

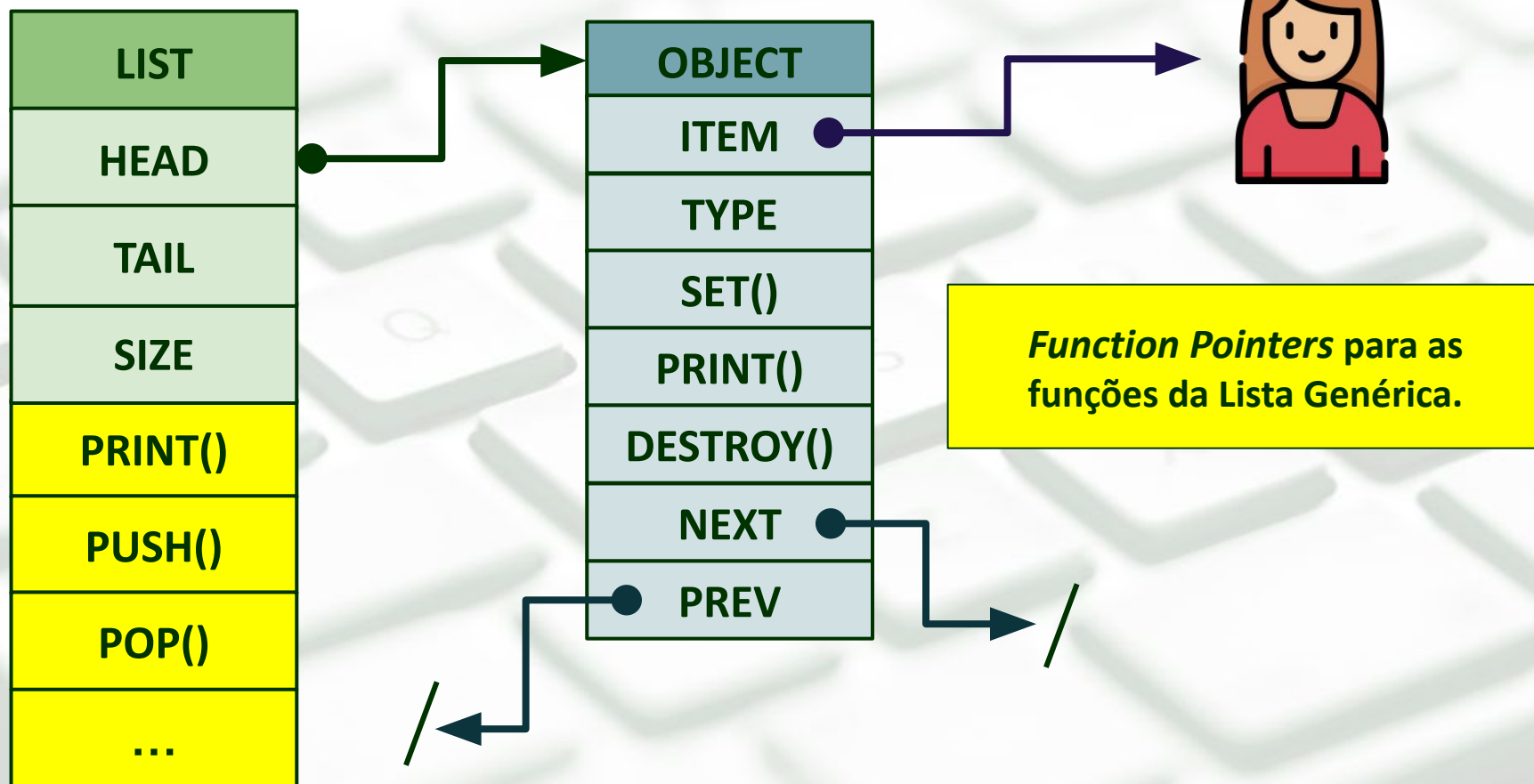
■ Lista Genérica Heterogênea





Projeto de Implementação

■ Lista Genérica Heterogênea





■ Lista de Objetos Genéricos v2.0

Object obj = new(?)

<Criar um novo Objeto Genérico>

obj->set()

<Editar dados do Objeto>

obj->print()

<Imprimir dados do Objeto>

obj->destroy()

<Deletar Objeto da Memória>

List lst = new(List)

<Criar uma Lista Vazia>

lst->print()

<Listar todos Obj da Lista>

lst->enqueue()

<Adicionar Obj ao Fim da Lista>

lst->delete(index)

<Deletar Obj com na Pos. index>

lst->push()

<Adicionar Obj ao Início da Lista>

lst->get(index)

<Obter Objeto na Pos. index>

lst->pop()

<Remover primeiro Obj da Lista>

lst->clear()

<Limpar/Excluir todos Obj da Lista>



■ Requisitos Técnicos...

- Implementação de **Function Pointers** nas Structs das Entidades.
- Tipagem dinâmica de objetos (mesmo nome, entidades distintas).
- Lista poderá operar com Entidades Heterogêneas

```
int main(){
    List lst = new(List);
    Object obj = new(Disciplina);
    setDisciplina(obj,"ED2",3,80);
    list_push(lst,obj);
    obj = new(Professor);
    setProfessor(obj,"Adriano","BSI");
    list_push(lst,obj);
    Object obj = list_pop(lst);
    printProfessor(obj);
    destroy(obj);
    list_print(lst);
    list_clear(lst);
}
```

```
int main(){
    List lst = new(List);
    Object obj = new(Disciplina);
    obj->set(obj,"ED2",3,80);
    lst->push(lst,obj);
    obj = new(Professor);
    obj->set(obj,"Adriano","BSI");
    lst->push(lst,obj);
    Object obj = lst->pop(lst);
    obj->print(obj);
    obj->destroy(obj);
    lst->print(lst);
    lst->clear(lst);
}
```